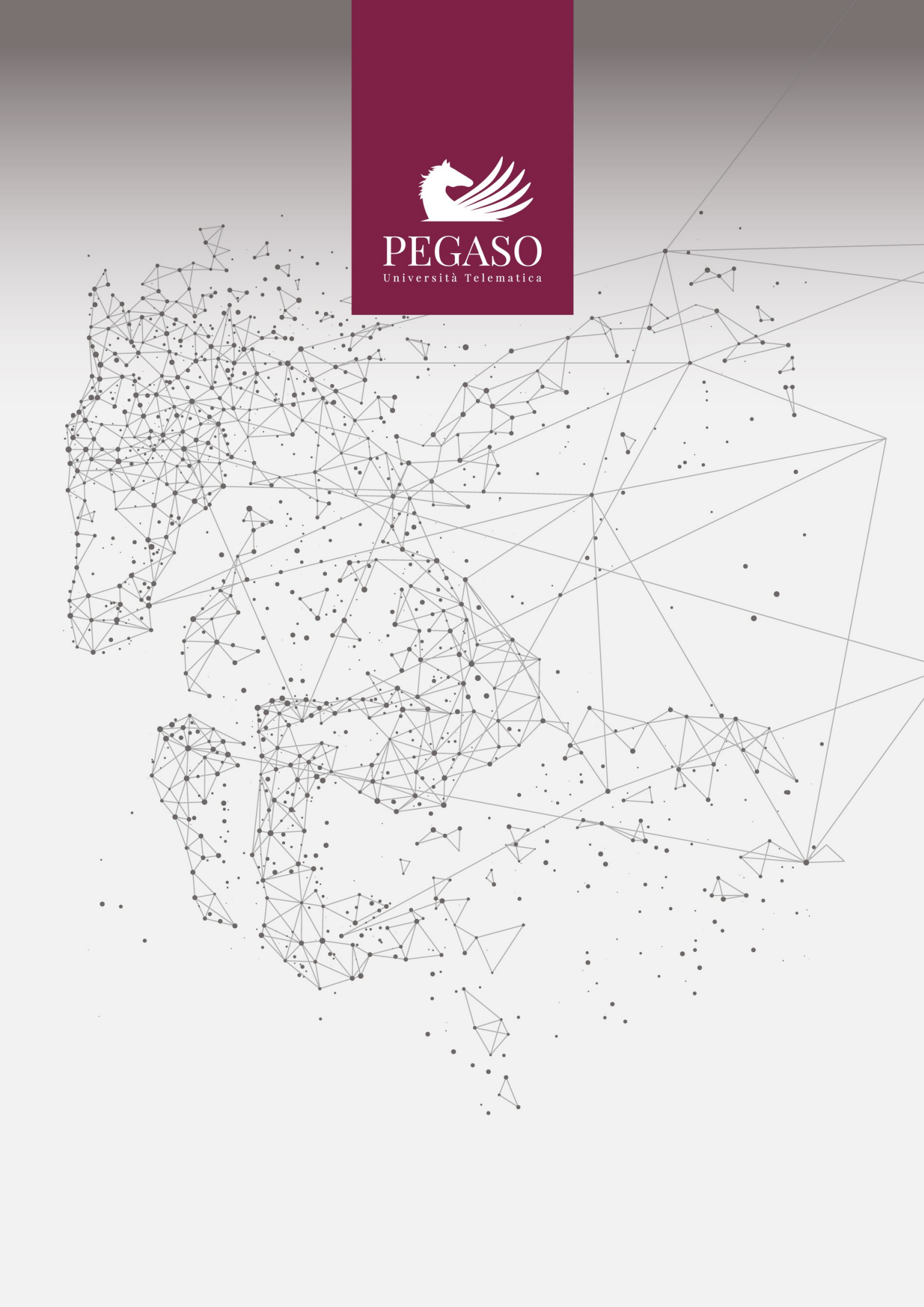

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. AGGIUNGERE I WIDGET AL FORM 4
3. RECUPERO E RESET DEI DATI DEL FORM 11
4. CLASSE APPLICAZIONE E SALVATAGGIO SU FILE CSV 12
BIBLIOGRAFIA 18

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presenta la procedura per la programmazione di un form. In particolare, si definisce una nuova classe che corrisponderà al form opportunamente progettato. Successivamente, si creano le sezioni del form e, per ciascuna di esse, si aggiungono i widget desiderati. Si presenta, poi, il codice sia per il reset del contenuto dei widget del form, sia per il recupero dei dati inseriti in ciascun widget di input. Infine, si illustra il codice per l'aggiunta del pulsante di salvataggio, per il relativo metodo di callback e per l'impostazione della status bar che fornirà all'utente importanti informazioni sul salvataggio dei dati inseriti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Aggiungere i widget al form

Quando si programma una GUI, è buona norma non "costruire" il form direttamente nella finestra principale dell'applicazione, ma "costruire" il ***form come oggetto a sè stante*** da inserire successivamente nella finestra principale. Ciò rende più semplice *mantenere un layout interessante* e permette *espansioni all'applicazione.*

Si parte, dunque, nuovamente dal *subclassing di tk.Frame* per costruire il form. Si procede con la definizione della **classe DataRecordForm.** La prima parte del codice della **classe DataRecordForm** è la seguente:

```
class DataRecordForm(tk.Frame):
    # Il form per i widgets

    def __init__(self, parent, *args, **kwargs):
        super().__init__(parent, *args, **kwargs)
```

Si crea, poi, una *struttura (**self.inputs**) per mantenere i riferimenti a tutti i widget di input che andremo ad utilizzare.* In particolare, si utilizza il **dizionario** *self.inputs* (inizialmente vuoto) in cui la ***chiave sarà il nome del widget di input:***

```
self.inputs = {}
```

Come da progetto, il **form** viene suddiviso in **sezioni** caratterizzate da *un'etichetta e un riquadro.* Per *ciascuna sezione*, infatti, si crea un **widget LabelFrame** e si aggiungono i ***widget LabelInput*** attraverso i seguenti passi.

2.1 Sezione "Informazioni sulle registrazioni"

Aggiungiamo un ***LabelFrame*** per la ***sezione*** "*Informazioni sulle registrazioni*" attraverso il seguente codice:

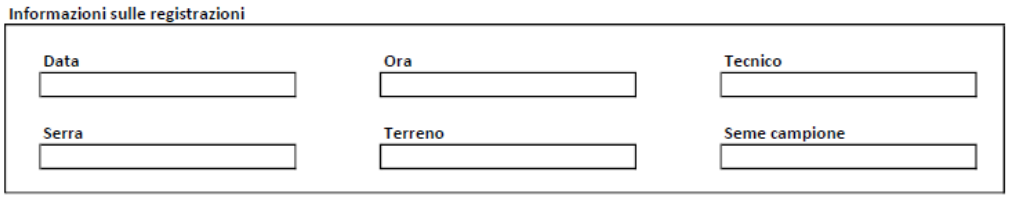
```
recordinfo = tk.LabelFrame(self, text="Informazioni sulle registrazioni")
```

dove ***text*** definisce il testo dell'etichetta del LabelFrame e il **widget recordinfo** verrà passato come ***widget parent*** a tutti i widget di input contenuti all'interno della sezione.

Il **modello** (***mockup***) da seguire della sezione "*Informazioni sulle registrazioni*" del form è il seguente:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



La corrispondenza tra ciascun campo della sezione e il tipo di widget che verrà utilizzato è riportata nella tabella seguente.

Campo	Tipo widget
Data	*ttk.Entry*
Ora	*ttk.Combobox*
Tecnico	*ttk.Entry*
Serra	*ttk.Combobox*
Terreno	*ttk.Combobox*
Seme campione	*ttk.Entry*

Si aggiungono alla prima linea del LabelFrame i **widget LabelInput** *Data, Ora* e *Tecnico* attraverso il seguente codice:

```
# linea 1
self.inputs['Data'] = LabelInput(
    recordinfo, "Data",
    input_var=tk.StringVar()
)
self.inputs['Data'].grid(row=0, column=0)
self.inputs['Ora'] = LabelInput(
    recordinfo, "Ora",
    input_class=ttk.Combobox,
    input_var=tk.StringVar(),
    input_args={"values": ["8:00", "12:00", "16:00", "20:00"]}
)
self.inputs['Ora'].grid(row=0, column=1)
self.inputs['Tecnico'] = LabelInput(
    recordinfo, "Tecnico",
    input_var=tk.StringVar()
)
self.inputs['Tecnico'].grid(row=0, column=2)
```

In particolare, ***Data*** e ***Tecnico*** sono solo dei semplici *input testuali (widget tk.Entry)* al cui costruttore (LabelInput) bisognerà passare ***parent*** (LabelFrame recordinfo), ***label*** e ***input_var.***

Per l'input ***Ora***, oltre ai parametri precedenti, si specifica anche una lista di valori possibili per l'orario che possono essere utilizzati per inizializzare il ***widget Combobox.***

Procediamo con la *seconda riga del LabelFrame* per la sezione «*Informazioni sulle registrazioni*». In questo caso sono presenti altri ***due widget Combobox*** e un ***widget Entry*** come si può osservare nel codice seguente:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
# linea 2
self.inputs['Serra'] = LabelInput(
    recordinfo, "Serra",
    input_class=ttk.Combobox,
    input_var=tk.StringVar(),
    input_args={"values": ["A", "B", "C", "D", "E"]}
)
self.inputs['Serra'].grid(row=1, column=0)
self.inputs['Terreno'] = LabelInput(
    recordinfo, "Terreno",
    input_class=ttk.Combobox,
    input_var=tk.IntVar(),
    input_args={"values": list(range(1, 21))}
)
self.inputs['Terreno'].grid(row=1, column=1)
self.inputs['Seme campione'] = LabelInput(
    recordinfo, "Seme campione",
    input_var=tk.StringVar()
)
self.inputs['Seme campione'].grid(row=1, column=2)
```

In particolare, i valori per ***Terreno*** corrispondono a una lista dei numeri da 1 a 20 ottenuta con la funzione *range*() il cui risultato viene trasformato successivamente in lista con la funzione *list*(). I valori possibili del **widget Serra** sono, invece, le lettere da A ad E.

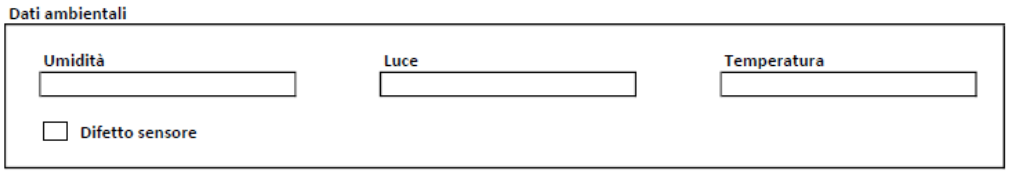
Una volta creati i widget di input dell'intera sezione, richiamiamo il metodo *grid*() su *recordinfo* per sistemarlo all'interno del Frame principale con il seguente codice:

```
recordinfo.grid(row=0, column=0, sticky=(tk.W + tk.E))
```

Procediamo, dunque, con le altre sezioni «*Dati ambientali*», «*Dati del terreno*» e «*Note*».

2.2 Sezione "Dati ambientali"

Il **modello** (***mockup***) da seguire della sezione "*Dati ambientali*" del form è riportato nella figura seguente.



La corrispondenza tra ciascun campo della sezione e il tipo di widget che verrà utilizzato è riportata nella tabella seguente.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Campo	Tipo widget
Luce	*ttk.Spinbox*
Umidità	*ttk.Spinbox*
Temperatura	*ttk.Spinbox*
Equipment fault	*Ttk.Checkbutton*

Per impostare la sezione "*Dati ambientali*" istanzieremo un nuovo ***LabelFrame*** denominato ***environmentinfo***. Aggiungiamo, attraverso il codice seguente, un nuovo **widget Spinbox** per il campo ***Umidità*** che specifica l'*intervallo valido* e la *quantità dell'incremento-decremento*. Si noti che la chiamata a *grid*() per il ***widget Umidità*** utilizza le coordinate di righe (0,0) proprio perché è iniziato un *nuovo oggetto parent* (***sezione*** del ***form***).

```
environmentinfo = tk.LabelFrame(self, text="Dati ambientali")
self.inputs['Umidità'] = LabelInput(
    environmentinfo, "Umidità (g/m³)",
    input_class=tk.Spinbox,
    input_var=tk.DoubleVar(),
    input_args={"from_": 0.5, "to": 52.0, "increment": .01}
)
self.inputs['Umidità'].grid(row=0, column=0)
```

Aggiungiamo come fatto per il campo Umidità, **due nuovi widget Spinbox** per i campi ***Luce*** e ***Temperatura*** con il seguente codice:

```
self.inputs['Luce'] = LabelInput(
    environmentinfo, "Luce (klx)",
    input_class=tk.Spinbox,
    input_var=tk.DoubleVar(),
    input_args={"from_": 0, "to": 100, "increment": .01}
)
self.inputs['Luce'].grid(row=0, column=1)
self.inputs['Temperatura'] = LabelInput(
    environmentinfo, "Tenmperature (°C)",
    input_class=tk.Spinbox,
    input_var=tk.DoubleVar(),
    input_args={"from_": 4, "to": 40, "increment": .01}
)
self.inputs['Temperatura'].grid(row=0, column=2)
```

Per il ***Checkbutton*** relativo al campo ***Equipment fault*** si utilizza una *BooleanVar* per memorizzare il suo valore. Si noti, nel codice seguente, anche l'utilizzo di *columnspan* al fine di far occupare al Checkbutton tutte e tre le colonne del **LabelFrame environmentinfo**.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
self.inputs['Equipment Fault'] = LabelInput(
    environmentinfo, "Equipment Fault",
    input_class=ttk.Checkbutton,
    input_var=tk.BooleanVar()
)
self.inputs['Equipment Fault'].grid(row=1, column=0, columnspan=3)
```

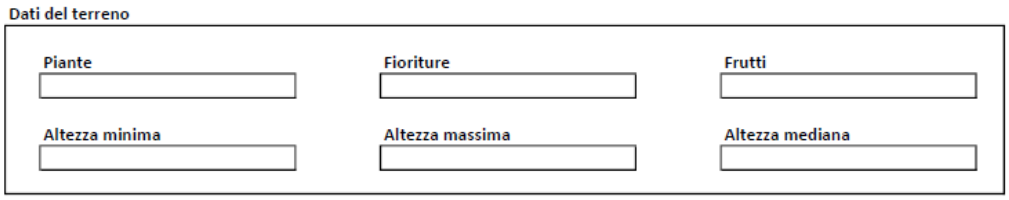
Una volta creati i widget di input, richiamiamo il metodo *grid*() su ***environmentinfo*** per sistemarlo all'interno del Frame principale come seconda sezione (*row=1*):

```
environmentinfo.grid(row=1, column=0, sticky=(tk.W + tk.E))
```

Procediamo con le altre sezioni «*Dati del terreno*» e «*Note*».

2.3 Sezione "Dati del terreno"

Il **modello** (***mockup***) da seguire della sezione "*Dati del terreno*" del form è:



La corrispondenza tra ciascun campo della sezione e il tipo di widget che verrà utilizzato è riportata nella tabella seguente.

Campo	Tipo widget
Fioriture	*ttk.Spinbox*
Frutti	*ttk.Spinbox*
Piante	*ttk.Spinbox*
Altezza massima	*ttk.Spinbox*
Altezza minima	*ttk.Spinbox*
Altezza mediana	*ttk.Spinbox*

Per la sezione "*Dati del terreno*", istanziamo un nuovo ***LabelFrame*** denominato ***plantinfo*** in cui verranno aggiunti **sei widget Spinbox** che specificano l'*intervallo valido* e la *quantità dell'incremento-decremento* ricordando che per i numeri interi non è necessario specificare tale quantità poiché di default è uguale a 1.

Si noti che, nel caso in cui non fosse possibile stabilire un valore massimo o un valore minimo per l'intervallo, è possibile utilizzare le stringhe ***infinity*** o ***-infinity.***

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
# sezione dati del terreno
plantinfo = tk.LabelFrame(self, text="Dati del terreno")

self.inputs['Piante'] = LabelInput(
    plantinfo, "Piante",
    input_class=tk.Spinbox,
    input_var=tk.IntVar(),
    input_args={"from_": 0, "to": 20}
)
self.inputs['Piante'].grid(row=0, column=0)
self.inputs['Fioriture'] = LabelInput(
    plantinfo, "Fioriture",
    input_class=tk.Spinbox,
    input_var=tk.IntVar(),
    input_args={"from_": 0, "to": 1000}
)
self.inputs['Fioriture'].grid(row=0, column=1)
self.inputs['Frutti'] = LabelInput(
    plantinfo, "Frutti",
    input_class=tk.Spinbox,
    input_var=tk.IntVar(),
    input_args={"from_": 0, "to": 1000}
)
self.inputs['Frutti'].grid(row=0, column=2)
```

```
self.inputs['Altezza minima'] = LabelInput(
    plantinfo, "Altezza minima (cm)",
    input_class=tk.Spinbox,
    input_var=tk.DoubleVar(),
    input_args={"from_": 0, "to": 1000, "increment": .01}
)
self.inputs['Altezza minima'].grid(row=1, column=0)
self.inputs['Altezza massima'] = LabelInput(
    plantinfo, "Altezza massima (cm)",
    input_class=tk.Spinbox,
    input_var=tk.DoubleVar(),
    input_args={"from_": 0, "to": 1000, "increment": .01}
)
self.inputs['Altezza massima'].grid(row=1, column=1)
self.inputs['Altezza mediana'] = LabelInput(
    plantinfo, "Altezza mediana (cm)",
    input_class=tk.Spinbox,
    input_var=tk.DoubleVar(),
    input_args={"from_": 0, "to": 1000, "increment": .01}
)
self.inputs['Altezza mediana'].grid(row=1, column=2)
```

Una volta creati i widget di input, richiamiamo il metodo *grid*() su ***plantinfo*** per sistemarlo all'interno del Frame principale come terza sezione (*row=2*):

```
plantinfo.grid(row=2, column=0, sticky=(tk.W + tk.E))
```

Procediamo con l'ultima sezione «*Note*».

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2.4 Sezione “Note”

Per la **sezione Note** non è necessario aggiungere un LabelFrame poiché è possibile aggiungere il ***widget LabelInput*** direttamente al form (DataRecordForm). Il **widget Text** prenderà come parametri di *larghezza* (***width***) e *altezza* (***height***) la dimensione del riquadro 75*10.

```
# sezione note
self.inputs['Note'] = LabelInput(
    self, "Note",
    input_class=tk.Text,
    input_args={"width": 75, "height": 10}
)
self.inputs['Note'].grid(sticky=tk.W, row=3, column=0)
```

```
import tkinter as tk
from tkinter import ttk

parent = tk.Tk()

tvar = tk.StringVar()
def swaptext():
    tvar.set('Mondo' if tvar.get() == 'Ciao' else 'Ciao')

my_button = ttk.Button(parent, textvariable=tvar, command=swaptext)

my_button.pack()


parent.mainloop()
```



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d’autore (L. 22.04.1941/n. 633).

3. Recupero e reset dei dati del form

Una volta terminato il form, è necessario scrivere il *codice* per **recuperare i dati da esso** al fine di essere elaborati dall'applicazione. Creiamo, quindi, un **metodo** che restituisce un ***dizionario con i dati del form.*** Come per gli *oggetti LabelInput*, tale metodo deve rispettare la convenzione Tkinter chiamandosi ***get***(). Il codice del metodo get() è il seguente:

```
def get(self):
    data = {}
    for key, widget in self.inputs.items():
        data[key] = widget.get()
    return data
```

Il codice è molto semplice in quanto «*cicla*» su tutti i **widget di input** e costruisce un **dizionario** chiamando il metodo **get**() su ciascuno dei widget stessi.

Di fondamentale importanza è anche la possibilità di ***resettare i dati inseriti nel form*** soprattutto dopo un salvataggio. Resettare i campi del forum vuol dire svuotarli (''). Il codice del metodo ***reset***() che come *get*() «*ciclerà*» sul dizionario dei widget di input ponendo ciascun widget a un valore vuoto, è il seguente:

```
def reset(self):
    # reset di tutti i testi contenuti nei widget di input
    for widget in self.inputs.values():
        widget.set('')
```

Per essere sicuri che l'applicazione si comporti in modo *consistente*, sarà necessario, poi, chiamare il metodo ***reset***() subito dopo il caricamento dell'applicazione, annullando tutti i valori di default di Tk. Per tale motivo come ultima riga del **costruttore __init__()** della ***classe DataRecordForm*** aggiungiamo la seguente istruzione:

self.reset()

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Classe applicazione e salvataggio su file CSV

Una volta creato il form è necessario costruire la **classe applicazione** che è *sottoclasse della classe Tk*. Nel costruttore si richiamerà il *costruttore della superclasse tk.Tk* passandogli gli argomenti come mostrato nel codice seguente.

```
class Application(tk.Tk):
    # finestra root dell'applicazione

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
```

Nel costruttore, attraverso il codice seguente si procede, poi, con l'impostazione della **stringa del titolo dell'applicazione** attraverso il metodo ***title***(). Si disabilita, inoltre, la possibilità di *ridimensionare la finestra* chiamando il metodo ***resizeable***() e passandogli il valore *False* sia per la larghezza (width) che per l'altezza (height).

```
self.title("Applicazione di inserimento dati ABQ")
self.resizable(width=False, height=False)
```

Aggiungiamo anche un *oggetto Label* nella parte alta della finestra dell'applicazione che mostra il **nome dell'applicazione** utilizzando un *font* più grande di quello normale. Si noti che non si specifica *column* (oltre alla riga row) perché il layout dell'applicazione principale è caratterizzato da una sola colonna (valore di default pari a 0).

```
ttk.Label(
    self,
    text="Applicazione di inserimento dati ABQ",
    font=("TkDefaultFont", 16)
).grid(row=0)
```

Sempre al *costruttore di Application* aggiungiamo un oggetto di tipo **DataRecordForm** come segue:

```
self.recordform = DataRecordForm(self)
self.recordform.grid(row=1, padx=10)
```

Con *padx=10* vengono aggiunti ***10 pixel di padding*** (riempimento) a sinistra e a destra nella chiamata a *grid*(). Questo aggiunge un piccolo spazio bianco intorno ai lati del form rendendo il form stesso più leggibile.

Si aggiunge, ora, un ***pulsante di salvataggio dei dati*** con il codice seguente:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
self.savebutton = ttk.Button(self, text="Salva", command=self.on_save)
self.savebutton.grid(sticky=tk.E, row=2, padx=10)
```

Il parametro ***command*** viene impostato con il riferimento al metodo ***on_save***() che andremo a scrivere tra poco. È convenzione chiamare i metodi o le funzioni di **callback di un evento della GUI** usando il formato ***on _eventname*** dove *eventname* è la stringa che descrive l'evento che ha generato la *callback*.

Infine, si aggiunge la ***status bar*** nella parte bassa del form con il seguente codice:

```
self.status = tk.StringVar()
self.statusbar = ttk.Label(self, textvariable=self.status)
self.statusbar.grid(sticky=(tk.W + tk.E), row=3, padx=10)
```

In particolare, si crea una variabile di tipo *StringVar* denominata ***status*** che viene utilizzata per il parametro *textvariable* del **widget ttk.Label *statusbar***. Per aggiornare lo stato in qualsiasi punto della classe, sarà necessario richiamare il metodo *set*() sulla variabile *self.status.*

Il codice completo del ***costruttore di Application*** sarà, dunque:

```
def __init__(self, *args, **kwargs):
    super().__init__(*args, **kwargs)

    self.title("Applicazione di inserimento dati ABQ")
    self.resizable(width=False, height=False)

    ttk.Label(
        self,
        text="Applicazione di inserimento dati ABQ",
        font=("TkDefaultFont", 16)
    ).grid(row=0)

    self.recordform = DataRecordForm(self)
    self.recordform.grid(row=1, padx=10)

    self.savebutton = ttk.Button(self, text="Salva", command=self.on_save)
    self.savebutton.grid(sticky=tk.E, row=2, padx=10)

    # status bar
    self.status = tk.StringVar()
    self.statusbar = ttk.Label(self, textvariable=self.status)
    self.statusbar.grid(sticky=(tk.W + tk.E), row=3, padx=10)

    self.records_saved = 0
```

Quando l'utente cliccherà sul **pulsante di salvataggio**, verrà attivata la seguente catena di eventi:

1. verrà aperto il file *abq_data_record_DATACORRENTE.csv;*
2. se il file non esiste verrà creato e il nome dei campi verrà scritto nella prima linea;
3. il dizionario dei dati sarà recuperato ***dall'oggetto DataRecordForm*** (contenente tutte le sezioni del form);
4. il dato viene formattato come una **riga CSV** (*record*) e viene ***aggiunto in append*** al file;
5. il form viene *resettato* e *all'utente viene comunicato che i dati del form sono stati salvati.*

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per poter eseguire tutti i passaggi è necessario introdurre alcune ***librerie di Python*** utili per i nostri scopi.

```
def on_save(self):
    #gestione del click del pulsante di salvataggio

    datestring = datetime.today().strftime("%Y-%m-%d")
    filename = "abq_data_record_{}.csv".format(datestring)
    newfile = not os.path.exists(filename)
```

Innanzitutto, si dovrà *recuperare una stringa corrispondente ad una data* per impostare il nome del file CSV. La *libreria* in questione si chiama ***datetime***. Si dovrà verificare, inoltre, *se un file esiste*. La *libreria* di aiuto sarà ***os***. Infine, dovremmo scrivere sul file CSV e per tale scopo verrà utilizzata la *libreria* ***csv*** della *standard library di Python.*

Si procede, quindi, con *l'importazione delle tre librerie* all'inizio del programma con il seguente codice:

```
from datetime import datetime
import os
import csv
```

All'interno della *classe Application*, definiamo il metodo ***on_save***(). Per ricavare la *data corrente* e impostarne il *formato* si può richiamare la seguente istruzione:

datestring = datetime.today().strftime("%Y-%m-%d")

Per utilizzare la data all'interno del nome del file si utilizza la seguente istruzione:

filename = "abq_data_record_{}.csv".format(datestring)

Per verificare se un file esiste già all'interno di una cartella, è possibile richiamare il metodo ***exists***() che restituirà un *valore booleano* (*True* se il file esiste, *False* se non esiste).

Il codice del metodo ***on_save***() finora è dunque:

La freccia rossa sul codice precedente indica che *newfile* sarà un valore booleano uguale al valore negato restituito dal metodo *exists*() ed indicherà la necessità (True) o meno (False) di un nuovo file.

Si recuperano, poi, i dati dall'oggetto ***DataRecordForm*** attraverso l'invocazione del metodo ***get***():

data = self.recordform.get()

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Acquisiti i dati, si *apre il file in append* e si crea un ***oggetto csv.DictWriter*** utilizzando il ***gestore del file*** *fh*. Ciò permetterà di scrivere i dizionari relativi ai dati all'interno del **file CSV** facendo corrispondere le **etichette della riga di testa** del CSV con le **chiavi del dizionario** attraverso il campo ***fieldnames*** del **costruttore di DictWriter.**

```
with open(filename, 'a') as fh:
    csvwriter = csv.DictWriter(fh, fieldnames=data.keys())
```

Se il file non esiste (***newfile*** a *True*) dobbiamo scrivere i *nomi dei campi* nella prima riga attraverso la chiamata al metodo ***writeheader***(). Infine, si scrivono i dati del dizionario in una nuova riga usando il metodo ***writerow***() dell'oggetto **DictWriter**.

Al termine del blocco iniziato con la keyword ***with***, il *file* verrà *chiuso e salvato* automaticamente. Il codice precedente viene, dunque, integrato nel seguente modo:

```
with open(filename, 'a') as fh:
    csvwriter = csv.DictWriter(fh, fieldnames=data.keys())
    if newfile:
        csvwriter.writeheader()
    csvwriter.writerow(data)
```

A seguito del *click* sul *pulsante di salvataggio*, anche se l'operazione di salvataggio stessa va a buon fine, non viene, però, visualizzato nessun messaggio in merito alla richiesta dell'utente.

Andremo, quindi, a inserire una ***notifica*** nella *status bar* in cui si comunica all'utente:

1. che il **record** (insieme dei dati inseriti nel form) **è stato salvato**;
2. **quanti record sono stati salvati** nella *sessione corrente*.

Per *inizializzare il numero di record salvati* nella sessione corrente è necessario impostare a zero una *variabile* nel **costruttore di Application**:

self.records_saved = 0

Si aggiungono, quindi, nel codice del metodo **on_save**() le seguenti istruzioni per tenere il conto dei record salvati in ciascuna sessione e comunicare tale numero all'utente:

```
self.records_saved += 1
self.status.set(
    "{} records saved this session".format(self.records_saved))
self.recordform.reset()
```

Il codice completo del **metodo *on_save***() di Application sarà, dunque:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def on_save(self):
    #gestione del click del pulsante di salvataggio

    datestring = datetime.today().strftime("%Y-%m-%d")
    filename = "abq_data_record_{}.csv".format(datestring)
    newfile = not os.path.exists(filename)

    data = self.recordform.get()

    with open(filename, 'a') as fh:
        csvwriter = csv.DictWriter(fh, fieldnames=data.keys())
        if newfile:
            csvwriter.writeheader()
        csvwriter.writerow(data)

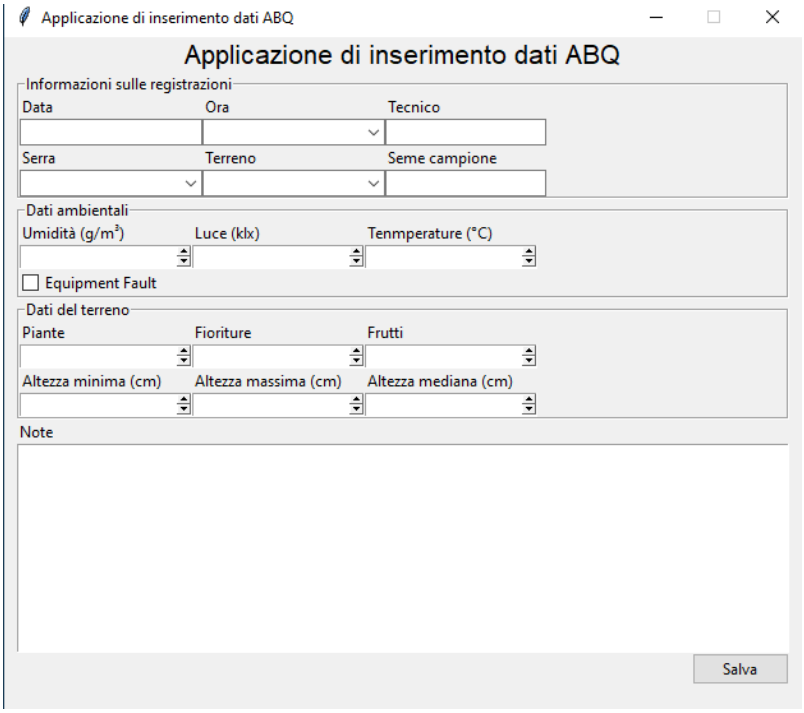
    self.records_saved += 1
    self.status.set(
        "{} records saved this session".format(self.records_saved))
    self.recordform.reset()
```

Infine, per avviare l'applicazione si richiamano le seguenti istruzioni:

```
if __name__ == "__main__":

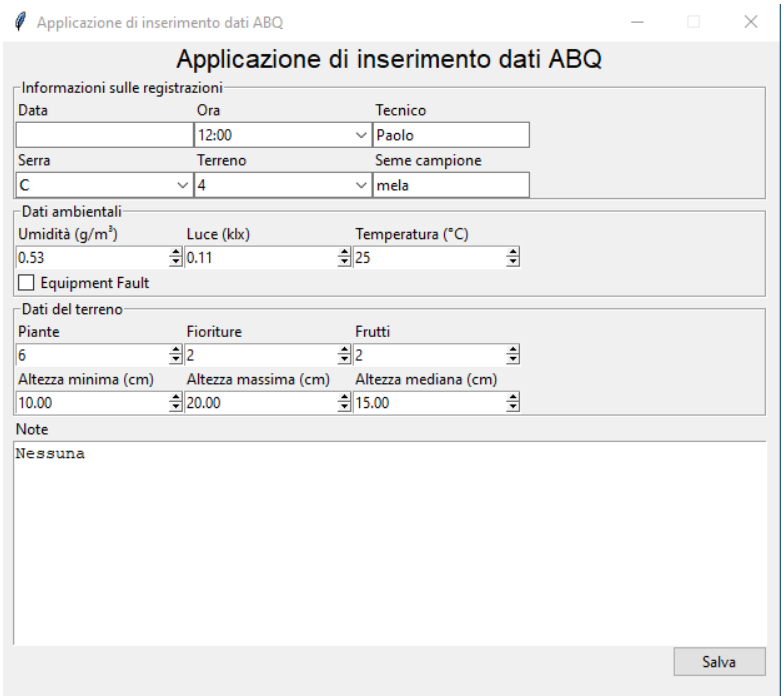
    app = Application()
    app.mainloop()
```

Nella figura seguente viene riportato il form visualizzato a schermo dopo l'esecuzione del codice:



Nella figura seguente, viene riportato un esempio di compilazione del form.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Nella figura seguente, viene riportato il messaggio della *status bar* che, a seguito del click sul pulsante "Salva" e conseguente salvataggio dei dati precedenti, mostra il numero di record salvati in una sessione.



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Moore A. D. (2018). Python GUI Programming with Tkinter. Packt

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).